System Architecture & Services Relationship

Project: Cloud Log Monitoring System

Course: CS 623 - Cloud Computing

Document: Architecture Diagram & Services Relationship

Team Members:

Shreya Kumari (NET ID: fd6317)

Richa Sapre (NET ID: ee9498)

1. High-Level Architecture Diagram

```
╔═══════════════════════════════════════════════════════════════════════╗
║                         CLOUD LOG MONITORING SYSTEM
╒                            Service Architecture
╚═══════════════════════════════════════════════════════════════════════╝

                              ┌──────────────────────────┐
                              │    CLIENT APPLICATIONS   │
                              │  (Web Servers, APIs,     │
                              │   Microservices)         │
                              └──────────────────────────┘
                                          │
                                          │
                                          │ HTTP POST /api/logs
                                          │ (JSON payload)
                                          ▼
╔═══════════════════════════════════════════════════════════════════════╗
║  GOOGLE CLOUD PLATFORM
╒═══════════════════════════════════════════════════════════════════════╗
╒
╒     ┌─────────────────────────────────────────────────────────────────
╒     │                         CLOUD RUN (api.py)
╒     │   ┌─────────────────────────────────────────────────────────────
╒     │   │  • Containerized Flask REST API
╒     │   │  • Endpoints: POST /api/logs, GET /api/logs, GET /api/stats
```

```
│   • Auto-scales based on traffic (0 to N instances)
│   • HTTPS endpoint with managed SSL
└──────────────────────────────────────────────────────────────────────────────
                                    │
                                    │
                                    │ Publish Message
                                    ▼
┌──────────────────────────────────────────────────────────────────────────────
│                        CLOUD PUB/SUB (logs-topic)
│   • Asynchronous message queue
│   • Decouples log ingestion from processing
│   • At-least-once delivery guarantee
│   • Messages retained for 7 days
└──────────────────────────────────────────────────────────────────────────────
                                    │
                                    │
                                    │ Push Subscription (HTTP Trigger)
                                    ▼
┌──────────────────────────────────────────────────────────────────────────────
│                      CLOUD FUNCTIONS (function.py)
│   • Serverless event processor
│   • Triggered automatically by Pub/Sub
│   • Processes and transforms log data
│   • Sends email alerts for ERROR/CRITICAL
└──────────────────────────────────────────────────────────────────────────────
                                    │                          │
                                    │                          │
                                    │ Store                    │ SMTP
                                    ▼                          ▼
┌──────────────────────────────────────┐   ┌──────────────────────────────────
│          FIRESTORE                   │   │          GMAIL SMTP
│   • NoSQL document database          │   │   • Email notifications
│   • Collection: 'logs'               │   │   • Triggered for ERROR/
│   • Auto-scaling storage             │   │     CRITICAL severity
│   • Real-time updates                │   │   • App Password auth
└──────────────────────────────────────┘   └──────────────────────────────────
                                    │
                                    │
                                    │
                                    │ Query (GET /api/logs)
                                    ▼
┌──────────────────────────────────────────────────────────────────────────────
│                          WEB DASHBOARD
│   React 18 + TypeScript + Tailwind CSS
│
│   Components:
```

```
│  │  • Header (navigation, refresh controls)          │    │
│  │  • StatsCards (metrics display)                   │    │
│  │  • TrendsChart (Recharts visualization)           │    │
│  │  • Filters (severity, source, date range)         │    │
│  │  • LogTable (sortable data grid)                  │    │
│  │  • AlertsPanel (custom alert rules)               │    │
│  │  • AlertToaster (browser notifications)           │    │
│  │                                                    │    │
│  │  Features:                                         │    │
│  │  • Auto-refresh every 10 seconds                  │    │
│  │  • Browser push notifications                      │    │
│  │  • Webhook integration (Slack)                     │    │
│  └────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

2. Service Relationship Matrix

The following table shows how each service connects to and communicates with other services:

From Service	To Service	Connection Type	Data Transferred
Client Applications	Cloud Run API	HTTPS REST	Log entries (JSON)
Cloud Run API	Pub/Sub	Google Client Library	Log messages
Cloud Run API	Firestore	Google Client Library	Query requests/responses
Pub/Sub	Cloud Functions	Push Subscription (HTTP)	Base64-encoded messages
Cloud Functions	Firestore	Google Client Library	Processed log documents
Cloud Functions	Gmail SMTP	SMTP (port 587)	Alert email content
Web Dashboard	Cloud Run API	HTTPS REST (Fetch API)	Log queries, new logs
Web Dashboard	Browser	Notification API	Push notifications
Web Dashboard	Slack (optional)	Webhook (HTTPS POST)	Alert payloads

3. Data Flow Sequence

Step 1: Log Ingestion
Client Application → POST /api/logs → Cloud Run API
Data: JSON with severity, message, source, timestamp

Step 2: Message Publishing
Cloud Run API → Publish → Pub/Sub Topic (logs-topic)
Data: Serialized log entry as message payload

Step 3: Event Processing

Pub/Sub → HTTP Push → Cloud Function (process-log-function)

Data: Base64-encoded message in Pub/Sub envelope

Step 4: Storage

Cloud Function → Add Document → Firestore (logs collection)

Data: Processed log with server timestamp

Step 5: Alerting (Conditional)

Cloud Function → SMTP → Gmail → User Email

Condition: severity == ERROR or CRITICAL

Step 6: Visualization

Web Dashboard → GET /api/logs → Cloud Run API → Firestore

Data: Array of log records with filtering/pagination

4. Service Details

4.1 Google Cloud Run

Property	Value
Service Name	log-monitoring-api
Region	us-central1 / us-west2
Container	Python 3.10 + Flask
Memory	512 MB
CPU	1 vCPU
Scaling	0-100 instances (auto)
Authentication	Allow unauthenticated

4.2 Google Cloud Pub/Sub

Property	Value
Topic Name	logs-topic
Subscription	Push to Cloud Function
Message Retention	7 days
Delivery	At-least-once

4.3 Google Cloud Functions

Property	Value
Function Name	process-log-function

Runtime	Python 3.10
Trigger	Pub/Sub (logs-topic)
Entry Point	process_log
Memory	256 MB
Timeout	60 seconds

4.4 Google Firestore

Property	Value
Database Mode	Native Mode
Collection	logs
Location	us-central
Indexes	severity, created_at (descending)

4.5 Web Dashboard (Frontend)

Property	Value
Framework	React 18.2.0
Language	TypeScript 5.6.2
Build Tool	Vite 5.4.9
Styling	Tailwind CSS 3.4.14
Charts	Recharts 2.12.7
Hosting	Firebase Hosting / Cloud Storage

5. Network & Security

5.1 Network Flow Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                   INTERNET                                   │
└──────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        │
                        ┌───────────────┴───────────────┐
                        │                               │
                        ▼                               ▼
          ┌──────────────────────────┐      ┌──────────────────────────┐
          │  Cloud Run API           │      │  Web Dashboard           │
          │  (HTTPS :443)            │      │  (HTTPS :443)            │
          │                          │      │                          │
          │  URL: *.run.app          │◀─────│  Hosted on Firebase      │
          └──────────────────────────┘  API │  or Cloud Storage        │
                        │             Calls └──────────────────────────┘
                        │
                        │
     ┌──────────────────────────────────────────┐
     │   GOOGLE CLOUD VPC                       │
     │                                          │
     │    ┌──────────────────────────┐          │
     │    │       Pub/Sub            │          │
     │    └──────────────────────────┘          │
     │                │                         │
     │                │                         │
     │    ┌───────────▼──────────────┐          │
     │    │   Cloud Functions        │──────────┼──────▶ Gmail SMTP
     │    └──────────────────────────┘          │        (smtp.gmail.com:587)
     │                │                         │
     │                │                         │
     │    ┌───────────▼──────────────┐          │
     │    │       Firestore          │          │
     │    └──────────────────────────┘          │
     │                                          │
     └──────────────────────────────────────────┘
```

5.2 Authentication & Authorization

Service	Authentication Method
Cloud Run API	Public (allow unauthenticated) - Protected by CORS

Pub/Sub	Service Account (Cloud Run identity)
Cloud Functions	Service Account (Pub/Sub push)
Firestore	Service Account (ADC - Application Default Credentials)
Gmail SMTP	App Password (environment variable)

6. External Services Links & Deployed URLs

The following table lists all external services used in this project and their corresponding URLs/documentation:

Service	Type	URL / Link	Purpose
Web Dashboard (Frontend)	Deployed Application	https://sacred-augury-478923-i9.web.app/	Live monitoring dashboard for viewing logs, stats, and alerts
Firebase Hosting	GCP Service	https://firebase.google.com/docs/hosting	Hosts the static React frontend with global CDN
Google Cloud Run	GCP Service	https://cloud.google.com/run/docs	Serverless container platform for REST API
Google Cloud Pub/Sub	GCP Service	https://cloud.google.com/pubsub/docs	Asynchronous message queue for decoupled processing
Google Cloud Functions	GCP Service	https://cloud.google.com/functions/docs	Serverless event-driven log processor
Google Firestore	GCP Service	https://cloud.google.com/firestore/docs	NoSQL document database for log

			storage
Gmail SMTP	External Service	https://support.google.com/mail/answer/7126229	Email alerts for ERROR/CRITICAL severity logs
React	Frontend Framework	https://react.dev/	UI library for building the dashboard
Tailwind CSS	CSS Framework	https://tailwindcss.com/	Utility-first CSS framework for styling
Recharts	Charting Library	https://recharts.org/	Data visualization for log trends
Vite	Build Tool	https://vitejs.dev/	Fast frontend build tool and dev server

6.1 Service Relationship Summary

How Services Connect:

1. **User** → Accesses https://sacred-augury-478923-i9.web.app/ (Firebase Hosting)
2. **Frontend** → Sends HTTP requests to → **Cloud Run API** (Backend REST service)
3. **Cloud Run API** → Publishes messages to → **Pub/Sub** (Message Queue)
4. **Pub/Sub** → Triggers → **Cloud Functions** (Event Processor)
5. **Cloud Functions** → Stores data in → **Firestore** (Database)
6. **Cloud Functions** → Sends alerts via → **Gmail SMTP** (Email Service)

7. Legend

Google Cloud Platform Services
Frontend (React)
External Services (Gmail)
Client Applications

End of Architecture Document